

Programming Project: Dice Boss Fighter
By: Joseph Thekveli & Deborah Maignan

Introduction:

Idea: The player will fight a monster boss.

The game is in an event-controlled loop until the player wins, meaning the boss's health is equal to zero, or the boss wins, meaning the player's health is equal to zero

ex. `while(player.getHealth() > 0 && boss.getHealth() > 0 && !victory){`

The user plays as a Player with 4 attacks, choosing one of their attacks to use on the boss.

Player attacks will hit or miss depending on the 12-sided die roll (which will be randomly generated) (a random number between 1 and 12, inclusive).

Example:

Choose your attack

-sword: deals 5 damage (needs to roll 2 or higher) (10 points)

-bow: deals 10 damage (needs to roll 4 or higher) (20 points)

-lightning magic: 20 damage (needs to roll 6 or higher) (50 points)

-fire magic: deals 30 damage (needs to roll 8 or higher) (100 points)

If the player chooses the bow and rolls a 5, then it will hit the boss.

If the player rolls a number lower than 4, then the attack misses, and the boss takes no damage.

The boss will have its own class and its own health bar. The boss has 1 attack (death slash), but the power of its damage will be determined by the 6-sided die roll.

(rolls a one): attack misses

(rolls a two or three): deals 5 damage

(rolls a four or five): deals 10 damage

(rolls a six): deals 15 damage

After each turn, the program will display the health of the boss and the player.

The boss has 30 health, and the player has 25 health.

Scoring is determined by a point system.

Successfully hitting the boss increases your score, and missing attacks doesn't increase the score.

You can get higher scores by taking risks.

Going back to the knight example, if the player chooses the bow, there is a higher chance of missing the attack than if they choose the sword. However, using the bow gives you more points than using the sword.

Basically, if you take a high risk, there is a chance of getting a high reward (a higher score).

After each victory against the boss, the scores will be recorded in an ArrayList.

If you lose, the score will not be recorded.

If you choose to access the scoreboard, then the page will display the top 3 scores.

Main Section:

Functionality:

- Attack choice(input): The method playerTurn starts with method playerTurnStart, which prompts the player to input their choice of attack through a scanner and returns the player's choice in the form of a string variable. Next, the player's choice variable is put as a parameter of the method playerChoice. playerChoice chooses the attack method to use based on the string input of the player's choice.
- Entity Class: The Entity class is the superclass of the Player and Boss classes. It contains the private health variable, which is used in both the Player and Boss classes. The Entity class also contains the getter method getHealth(which returns the health) and takeDamage(which allows entities to take damage by subtracting the health by a damage value). The Player and Boss classes use super in their constructors to access the health variable from their superclass.

Runtime Output pictures:

Attack Choice(playerTurn):

Input: bow

```
Welcome!
Enter Start, Quit, or Top three scores
start
Player's turn:
Choose your attack:
sword: deals 5 damage (needs to roll 2 or higher)(10 points)
bow: deals 10 damage (needs to roll 4 or higher)(20 points)
lightning: deals 20 damage (needs to roll 6 or higher)(50 points)
fire: deals 30 damage (needs to roll 8 or higher)(100 points)
bow
Player chose bow!
The attack hits! You gain 20 points!
-----
Boss used death slash for 15 damage.
Player Health:10
Boss Health:20
```

Entity Class:

```
public class Entity
{
    private int health;

    public Entity(int h){ // default constructor
        health = h;
    }
    /**
     * This program gets the health
     * @return health returns the health value
     */
    public int getHealth(){
        return health;
    }
    // This program allows entities (like the boss or player) to take damage
    public void takeDamage(int damage){
        health = health - damage;
    }
}
```

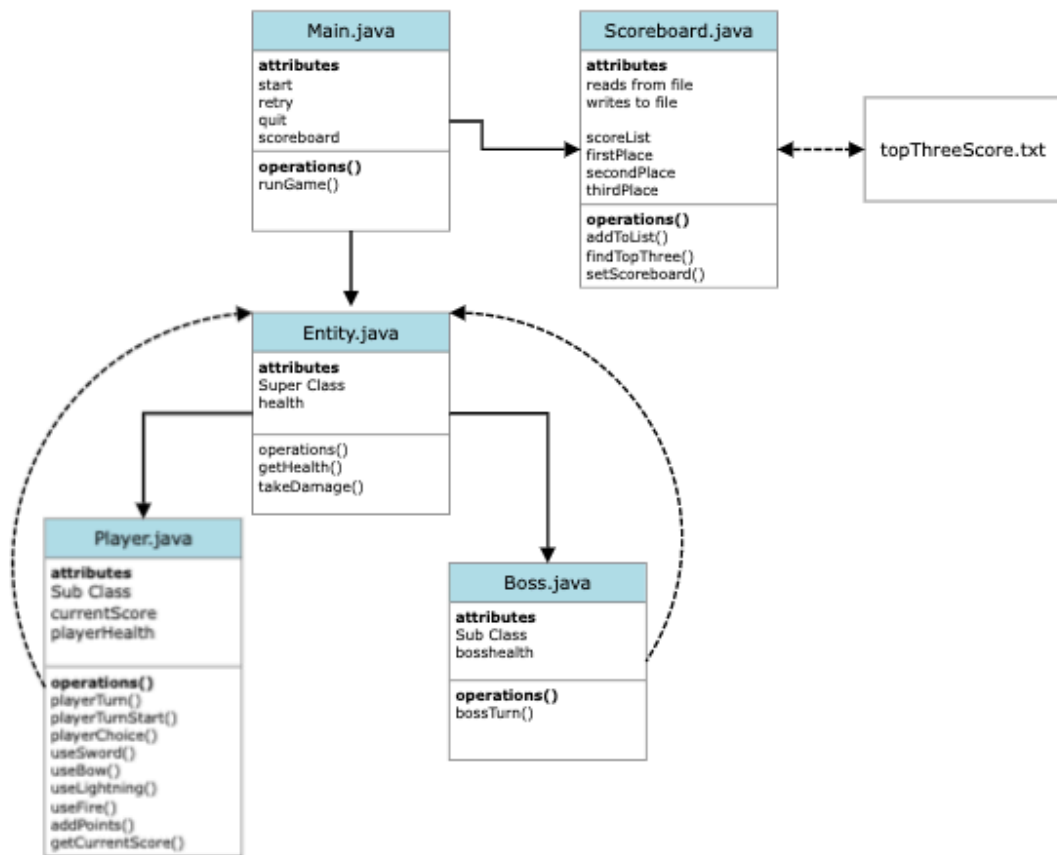
Accomplishments/Challenges:

- **runGame(how it worked):** The method `runGame` in the `Main` class was quite challenging to work with at first. When the method was called, it created a player object with 25 health, a boss with 30 health, a score variable of 0, and a boolean variable to determine the victory. The boolean is set to false by default. The while loop runs the player and boss turns as long as the player's and boss's health are above zero and the boolean is false. If the boss's health gets to zero, then the boolean is set to true, and it exits the while loop. The final score is added up and then put on the scoreboard. It then returns a boolean variable to determine the victory. The problem was that I couldn't keep declaring `Scoreboard` each time a new game was run, and I was already using `Scoreboard` in the main method to get the top 3 scores. To solve this, I made `runGame` return the score as an integer rather than a boolean. The score will return zero by default if the player loses. In the main method, the score will be added to the score list if the score is not zero. So I only had to declare the `Scoreboard` once in the main method, and I could still add and organize the score.
- **Scorelist(findTopThree):** The method `findTopThree` took some time to figure out. One of the problems with it was that it would occasionally duplicate the same score twice whenever I displayed the top 3 scores (for example: 120, 100, 100). I needed to find a more reliable way to organize the top 3 scores so they wouldn't display duplicate scores. My new idea was to organize the `ArrayList` of scores in descending order (greatest to least) and then get the first, second, and third place scores easily. I started the method by declaring a boolean variable to see if it was sorted. A while loop checks to see if the `ArrayList` is sorted or not. In the while loop, a for loop iterates through the `ArrayList` of scores, and if the value in the current index is smaller than the value in the next index, then it swaps those values in the `ArrayList` and sets the boolean variable to false. Before the for loop, the boolean variable is set to true. This must be so that if the `ArrayList` is not sorted, then the variable is set to false. This continues until the `ArrayList` is fully organized, and then the first, second, and third place variables are set to the first three indexes in the `ArrayList`, respectively. Now, the top 3 scores list doesn't contain any duplicate scores when it is displayed.

Classes:

- `Main.java`: where the user inputs actions (Start, Retry, Quit, Scoreboard)
- `Entity.java`: superclass of Player & Boss, (health)
- `Player.java`: contains player data (attacks, current score),
- `Boss.java`: contains boss data (attacks)
- `ScoreBoard.java`: contains an `ArrayList` of scores, calculates the top 3.

UML Diagram:



Interfaces:

- Attack choice: choose one of the 4 attacks for the turn (sword, bow, lightning, fire)
- Choose to quit (to main menu) or try again
- Option to see the top 3 scores

Files:

- topThree: contains the top 3 current scores (text file)

Conclusion:

Our program runs a game of probability, in an event-controlled while loop, between a player, with 4 attacks that hold different damage levels, and a boss, that has one attack but can administer different damage levels. We used 5 different Classes: **Main.java**, **Entity.java**, **Player.java**, **Boss.java**, and **Scoreboard.java**, including 2 that were inherited from a superclass (Superclass: **Entity.java** Subclasses: **Player.java**, and **Boss.java**), to make the game run altogether. The front end of our game is text-based, so the player interacts by sending inputs and reading the output. We used error handling in the **Player.java** class to catch any user's incorrect input. Our ArrayList, called `scoreList`, was used to add and get the final scores when prompted by the user input. Our file handling was in the **Scoreboard.java** class. Although it was difficult to get everything functioning properly, we're satisfied with how the program works.